Regenerative Autoencoders: Generation and Regeneration through Iterative Denoising

Alejandro Hernandez Ruiz

Revised draft, version 0.3 — October 29, 2026

Abstract

Denoising models learn to correct corrupted data toward a desired state. Neural cellular automata use the biological term regeneration *for recovery after an already developed pattern is damaged. Building on this viewpoint and the autoencoding lineage of Neural Cellular Automata Manifold, we present Regenerative Autoencoders (ReGAE): a family of generative denoising models that directly predict corrected states and reuse their outputs as subsequent inputs. An encoder supplies a latent representation in the conditional form; an unconditional variant retains the denoiser without that code. Neither variant requires an iteration index or noise-level input. For fixed conditioning, one learned correction map therefore serves both generation from noise and renewed restoration after perturbation. We interpret this construction geometrically as a time-independent state update, and compare it with time-indexed diffusion and flow models while distinguishing state, noise, and velocity prediction. The resulting landscape intuition does not assume a learned scalar energy or guaranteed convergence. A proof of concept on Pandora's Cluster composes two ReGAE instances to learn image samples and spatial relationships among their representations. The existing evidence motivates a broader discussion of dataset modeling, recursive architectures, and compression.*

1. Introduction

Generation constructs a sample from an initial state; repair resumes that construction after the sample has been disturbed. The term *regeneration* comes here from the biological motivation of neural cellular automata. Growing NCA distinguishes growth from a seed, persistence of the resulting pattern, and recovery after damage [11]. We adopt regeneration in this behavioral sense, rather than as a separate category of probabilistic model or a capability exclusive to cellular automata.

Neural Cellular Automata Manifold (NCAM) supplies the autoencoding lineage: an encoder selects a cellular growth program capable of reconstructing a target image [7]. Regenerative Autoencoders (ReGAE) retain the idea of a reusable reconstruction process while replacing the cellular decoder with a shape-preserving denoiser. The denoiser acts on a corrupted version of the desired state and directly predicts a corrected state. Its output becomes the input to the next update. Starting with noise gives generation; starting with a disturbed reconstruction gives regeneration. In the conditional form, a latent code identifies the intended reconstruction throughout either process.

Neither an iteration index nor a corruption-strength label is supplied to the ReGAE denoiser. At fixed conditioning, the same state is therefore handled by the same learned map wherever it occurs in a trajectory. This motivates viewing generation and repair as movement under one correction field, learned across a range of corruptions. The interpretation is related to diffusion and flow models, but their prediction targets, dependence on time, and sampling rules need to be separated carefully. A single map does not by itself establish a scalar optimization landscape or a globally attracting solution.

The broader question motivating this work is what it takes to learn, or *memorize*, a dataset. Calling a model a memorizer does not explain how that memory is organized, which properties it retains, or how it makes them available for generation. A dataset contains information about individual samples and, potentially, relationships among them. Learning to reconstruct a sample addresses one part of this problem. Learning the distribution or organization of its representations addresses another. These are useful questions even before asking whether a system extrapolates beyond its training data.

ReGAE provides a reusable modeling component for investigating those questions. An encoder can concentrate information about a particular sample into a code, while a conditional denoiser learns how to recover the sample from different corrupted states. Removing the encoder and its conditioning pathway gives an unconditional member of the family: a denoiser whose evolving state supplies its context. Such a model can operate directly on data or on representations produced by another model. Conditional and unconditional ReGAE instances can therefore be composed,

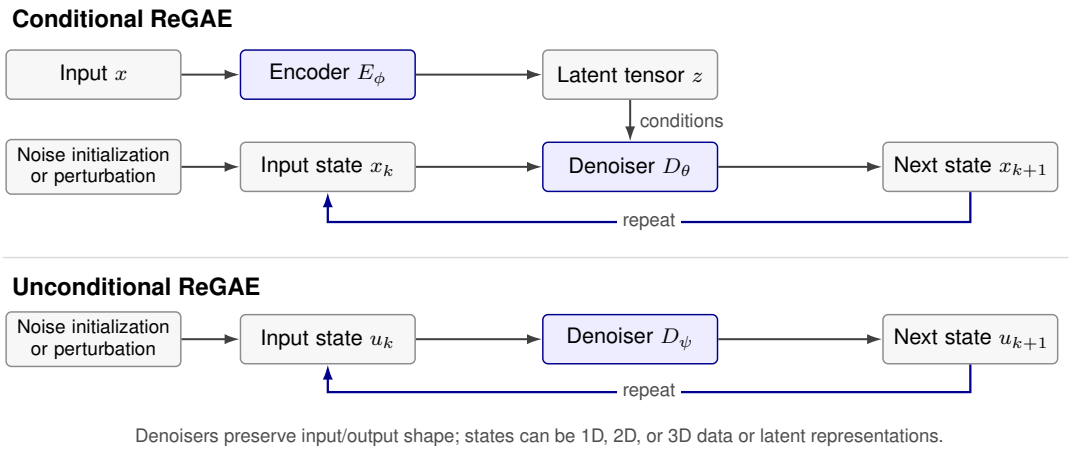


Figure 1. Conditional and unconditional members of the ReGAE family. Both denoisers return a state with the same shape as their input, allowing the output to be fed back for successive updates. Generation initializes the state with noise; regeneration re-enters the same loop with a perturbed state. In the conditional model, an encoder supplies a latent tensor that is retained during regeneration. The unconditional model has no encoder-derived conditioning and can model data or learned representations. The two rows illustrate variants, not mandatory levels of one hierarchy.

without requiring a particular number of levels or a particular kind of dataset.

The experiments presented here use a large image of Pandora's Cluster as an example dataset. An image-patch autoencoder learns local appearance, while a second, unconditional ReGAE learns the spatial organization of the resulting patch codes. We call this latent-space instance the *Gencoder*. Predicting codes for a slightly displaced view couples regeneration to a spatial transition. This arrangement is appropriate because spatial relationships are an important part of the information in this dataset. It is an application of the family, rather than its definition.

A different dataset would suggest a different decomposition. For a collection such as CIFAR-10, the first autoencoder could represent each complete image. A subsequent model could learn the distribution of image codes or, with suitable inputs and supervision, relationships between image and class representations. There would be no reason to impose the spatial arrangement used for Pandora's Cluster. This example illustrates the intended scope; it is not an additional experiment reported here.

The defining design choice is thus an iteratively reusable, shape-preserving denoiser, with optional latent conditioning. Its internal architecture, data dimensionality, reconstruction loss, and placement within a hierarchy are configurable. We describe this general construction first, then use the astronomical-image proof of concept to examine the distinction between sample reconstruction and modeling relationships among samples. The discussion connects regeneration to denoising dynamics, architectural reuse, and compression, while keeping the broader proposal distinct from the evidence supplied by the existing experiments.

2. Related Work

Regenerative neural dynamics. Growing Neural Cellular Automata distinguish growth, persistence, and regeneration, with damage during training strengthening recovery [11]. Neural Cellular Automata Manifold (NCAM) extends cellular growth to an autoencoding setting through dynamic convolutions [7]. Its reported code-mutation experiments concern robustness of the encoding; they do not constitute a damaged-state regeneration evaluation. ReGAE continues this investigation through conditional denoising autoencoders and an unconditional variant that retains the regenerative operator without an encoder. Their state inputs and outputs occupy the same domain and have matching dimensions, enabling repeated application. The image implementation studied here uses U-Nets and attention beyond a strictly local cellular rule; these components and the optional latent hierarchy instantiate the broader construction.

Denoising as generative modeling. Bengio et al. [2] formulate generative Markov chains through alternating corruption and denoising, including walkback training to address spurious modes. Alain and Bengio [1] connect regularized reconstruction to density scores under specified assumptions. ReGAE permits different reconstruction objectives, including L_1, L_2, and learned-feature losses. The present image application uses a VGG-based *perceptive* loss, often termed a perceptual loss [9, 17], while its latent predictor uses L_1. These choices are application-specific; a score interpretation requires checking the corresponding corruption and loss assumptions.

Diffusion and continuous-time models. Diffusion probabilistic models learn to reverse a prescribed noise process [8]; DDIM permits deterministic sampling through a related non-Markovian construction [18]. Score-based SDEs yield a probability-flow ODE with matching time-marginal distributions under the corresponding score field [19]. Neural ODEs parameterize continuous-time derivatives and integrate them with differential-equation solvers [4]. Flow Matching learns a time-dependent velocity field along probability paths [10]. Chan's tutorial connects signal and noise prediction with numerical sampling, providing a useful account of these geometric relationships [3]. ReGAE directly predicts desired states without explicit noise-time conditioning. Sections 3.2 and 5.3 distinguish this update from noise prediction and velocity integration, without treating any one parameterization as universal to diffusion.

Autoencoding and latent generative structure. Diffusion Autoencoders (DiffAE) are particularly close to ReGAE's conditional autoencoding formulation [13]. They combine a learned semantic encoder with a conditional DDIM decoder, representing an image through a 512-dimensional nonspatial semantic vector and a spatial stochastic code. Both parts contribute to near-exact reconstruction. DiffAE also fits a latent DDIM prior over semantic vectors using a deep multilayer perceptron with skip connections. Latent diffusion models likewise perform generative denoising in an autoencoder's representation space [15]. In our Pandora's Cluster application, an unconditional ReGAE, the Gencoder, models spatial windows of patch codes and predicts displaced encodings. This spatial prediction hierarchy is one composition of the family, and differs from DiffAE's prior over image-level semantic vectors.

Learning from a single image. SinGAN learns multiscale patch distributions from one natural image [16]. Our image case study similarly examines local appearance and spatial organization, using encoded patches and displaced prediction. The setting makes the distinction between reconstructing a particular arrangement and generating familiar-looking content visible, without restricting ReGAE to single-image modeling.

Representation and compression. Minimum description length counts both model and data descriptions [14]. COIN stores an image through quantized coordinate-network weights [6], while language-model compression connects predictive distributions to explicit coding [5]. These perspectives inform the interpretation of ReGAE representations. An operational compression claim additionally requires accounting for precision, model storage, residual information, and reconstruction distortion.

3. The ReGAE Model Family

A conditional Regenerative AutoEncoder (ReGAE) combines a compressed representation with a denoising operator that can be applied repeatedly to a mutable state. The representation specifies what to reconstruct, while the state supplies the current reconstruction, partial observation, or noise initialization. An unconditional member omits the encoder and its conditioning, retaining the recurrent denoiser. We describe these variants independently of the dimensionality, network widths, loss, and sampling choices of the Pandora's Cluster case study.

3.1. State, encoding, and conditional update

Let $\mathcal{X}$ be a space of sample tensors and $\mathcal{Z}$ a latent tensor space. An encoder $E_\phi : \mathcal{X} \to \mathcal{Z}$ compresses a sample x into $z = E_\phi(x)$. Neither space has a prescribed dimensionality: the latent may be a vector or a structured tensor, with a bottleneck chosen for the task. Compression here describes the learned representation; its storage cost also depends on numerical precision and model parameters.

The conditional denoiser has the interface

$$D_\theta : \mathcal{X} \times \mathcal{Z} \to \mathcal{X}, \qquad x_{k+1} = D_\theta(x_k, z). \tag{1}$$

For a chosen configuration, its state input and output have the same dimensions, allowing the output to become the next input without a separate encoding step. The same construction accommodates one-, two-, or three-dimensional state layouts, using suitable operators for sequences, images, or volumetric tensors. This is an architectural interface, rather than a claim that all such domains have been evaluated here.

The latent remains fixed during a conditional trajectory. Parameters are also shared across updates: iteration changes the state, not the learned weights. Conditioning can enter through feature modulation or another compatible mechanism. The encoder is needed to obtain a code from an observed sample; repeated denoising subsequently uses that code directly.

3.2. Direct state prediction without a clock

The denoiser predicts the desired state itself. For a current state u and a fixed code z, define its induced displacement

$$\Delta_z(u) = D_\theta(u, z) - u. \tag{2}$$

Then the update can equally be written

$$x_{k+1} = x_k + \Delta_z(x_k) = D_\theta(x_k, z). \tag{3}$$

The network's output already supplies the next state; no separate integration formula is needed to turn a predicted derivative or noise estimate into that state. The displacement in Eq. (2) is an interpretation of the state prediction, not an additional trained output.

The index k counts evaluations but is not a network input. Neither is the corruption strength. For fixed z and parameters, the same u invokes the same correction regardless of when it occurs. The basic recurrence is therefore an *autonomous discrete map*: it has no explicit clock. A perturbed state can re-enter this map without first assigning it a position in a denoising schedule. This describes the interface, not guaranteed recovery from every possible state.

3.3. Learning a regenerative denoiser

Training presents a corrupted version of a clean sample and asks the denoiser to recover the sample given its encoding. Let $C_\alpha(x, \epsilon)$ denote a corruption operator, with severity α and random variable ϵ. One useful choice is the mixture

$$C_\alpha(x, \epsilon) = (1 - \alpha)x + \alpha\epsilon, \qquad 0 \le \alpha \le 1. \tag{4}$$

The noise distribution, severity distribution, and any spatial masking are configuration choices. Noise is treated as corruption of the desired final state, rather than as the regression target. Including both weak and strong corruption trains one map across a range of departures from that state, connecting refinement with reconstruction from noise. The conditional training objective is

$$\mathcal{L}_{\text{cond}} = \mathbb{E}_{x,\alpha,\epsilon}\left[\ell\big(D_\theta(C_\alpha(x, \epsilon), E_\phi(x)), x\big)\right]. \tag{5}$$

The reconstruction criterion ℓ may be an L1 loss, squared L2 loss, or feature-based *perceptive* loss, chosen according to the data and desired fidelity. A feature loss compares representations of the prediction and target; it does not define the ReGAE family. Likewise, a particular noise schedule or optimizer is an implementation choice.

Since α is hidden from the predictor, the map must use the observed state and code to resolve ambiguities among corruption levels. The objective covers the *sampled corruption distribution*; it does not establish reliable behavior throughout all of state space. Appendix A gives a conditional-expectation interpretation for the squared-error case.

Equation (5) trains an individual restoration step. Repeated application at inference is a separate use of that learned operator. Good one-step reconstruction does not alone establish convergence of every trajectory, preservation of all details, or recovery from arbitrary perturbations. Regeneration must therefore be assessed for the corruption types and update horizons of interest.

3.4. Generation and regeneration

Conditional generation starts from a noise state $x_0 \sim p_0$ and applies Eq. (1) for a chosen number of updates. When $z = E_\phi(x)$, this constructs a reconstruction associated with the encoded sample. Generating new samples through new codes additionally requires a way to choose those codes; the conditional autoencoder alone does not specify their prior distribution.

Regeneration uses the same update after an external perturbation. If $\widehat{x}$ is a current reconstruction and P describes damage or corruption, set

$$y_0 = P(\widehat{x}), \qquad y_{k+1} = D_\theta(y_k, z). \tag{6}$$

The unchanged code supplies a reference while the denoiser acts on the altered state. Noise injection or partial-state refresh between updates can also be used as sampling choices. These interventions change the trajectory without requiring a new encoder evaluation or parameter update. A stable reconstruction may behave as an approximate fixed point, but fixed-point existence, attraction, and universal convergence are not assumed by the definition.

3.5. Unconditional variant and architecture

An unconditional ReGAE omits the encoder and external latent code, retaining the state-to-state denoiser:

$$\begin{aligned} x_{k+1} &= D_\theta(x_k), \\ \mathcal{L}_{\text{uncond}} &= \mathbb{E}\left[\ell(D_\theta(C_\alpha(x, \epsilon)), x)\right]. \end{aligned} \tag{7}$$

Here *unconditional* means no separate conditioning code; the output still depends on its input state. Starting from noise gives a generative process shaped by the learned data regularities. Starting from an existing state instead uses its content as context. Without an external code, there is no independent encoded reference that identifies a particular original sample during recovery.

Conditioning on a code and conditioning on time are distinct choices. Both ReGAE variants described here are time-unconditioned; only the conditional variant receives z. The Gencoder's lack of a code should therefore not be confused with the lack of a clock shared by both variants.

Our default denoiser uses a configurable U-Net-based architecture. Its depth, widths, normalization, attention, positional features, and conditioning mechanism can be selected for the problem while preserving matching state-input and state-output dimensions. Multiplicative feature gating is one optional component. For example, the gate called *gravity* can apply $h \odot \gamma(g)$ to learned features, with a bounded or exponential γ. Such a gate changes feature gain; it is neither a required component of ReGAE nor a physical model of gravity. The precise choices used in the case study are documented separately.

3.6. Optional hierarchy and predictive targets

A hierarchy can apply the family to representations learned by another ReGAE. A full conditional first stage encodes samples or local units and provides a decoder for each code. Their codes can then be assembled into a latent field Z. An unconditional second-stage ReGAE, called the *Gencoder*, learns a state update $Z_{k+1} = G_\psi(Z_k)$. Decoding its generated codes through the first stage maps latent organization

back into the original data space. The two stages have separate learned parameters; a latent hierarchy is optional, rather than part of the definition of every ReGAE.

The restoration target can also be generalized to a same-shaped paired target. For paired states (u, v) and optional conditioning c, train

$$\mathcal{L}_{\text{pair}} = \mathbb{E}\left[\ell\big(D_\theta(C_\alpha(u, \epsilon), c), v\big)\right], \tag{8}$$

omitting c for an unconditional instance. Ordinary restoration takes $v = u$. A predictive extension instead uses a related displaced or subsequent state as v, so repeated updates may produce progression rather than convergence to one reconstruction. The Pandora experiment applies this extension to shifted latent fields. Its patch sizes, offset grids, overlap rules, and video sampler instantiate that particular hierarchy; they do not restrict the general ReGAE family.

4. Proof of Concept: Modeling Pandora's Cluster

4.1. From individual samples to dataset structure

The proof of concept asks what must be learned to represent one large, structured visual dataset. The source is a rendered RGB mosaic of Pandora's Cluster from the James Webb Space Telescope's NIRCam instrument [12], with recorded dimensions of $17{,}644 \times 13{,}422$ pixels, or 236.8 megapixels. We treat its image patches as samples and their spatial relationships as another aspect of the dataset to be modeled. The source is a display composite rather than raw photometric data; the task concerns visual representation and synthesis.

An autoencoder can reconstruct patches when their codes are provided, but this alone does not specify how those codes should be arranged across the full image. Keeping every source code at its original location supplies that information externally. Generating familiar-looking patches without this arrangement solves a different problem. The experiment separates these two learning tasks to investigate where the information needed for each resides.

4.2. Two instances of the ReGAE family

The first instance is a full, conditional ReGAE: an encoder maps an RGB patch to a code, and a denoiser reconstructs the patch from a corrupted pixel state conditioned on that code. In this implementation, each 32×32 patch is represented by 256 latent values. The code dimensions and patch size are choices for this experiment, not properties of ReGAE in general.

The image reconstruction objective uses a VGG-based *perceptive* loss [9, 17]. Feature agreement is used to favor visual similarity that better reflects the intended human viewing experience than strict pixel matching alone. This choice does not define the model family: an L_1 or L_2 reconstruction objective could be used instead. The pretrained feature network supplies an external visual prior, and the available experiments do not compare these loss choices.

The second instance is an unconditional ReGAE over latent fields. It omits the encoder and external conditioning pathway and reuses the denoiser architecture family with separate weights. Its input is a spatial window of patch representations, and its output has the same shape. A 32×32 latent window can be decoded into a 1024×1024 image region. Here, *unconditional* means that no separate code identifies which window to produce; the current latent field still provides content and spatial context.

To encourage prediction throughout a window, the target codes are computed from image crops displaced by a fraction of the patch size. The pixel support of every target code changes, even though the underlying image regions overlap substantially. This discourages solving the transition by copying most latent tokens and filling only a new border. The shift is a task-specific extension of denoising: the Gencoder is trained to restore a clean representation of the *next view*, rather than to keep the same view fixed. Exact grid construction, losses, and schedules appear in Appendix B.

4.3. Recurrent synthesis and regeneration

For a conditional ReGAE, a code can initialize reconstruction from noise, and the resulting pixel state can be passed through the denoiser repeatedly. Holding that code fixed also provides a target-specific cue when restoring a perturbed reconstruction. The Pandora implementation trains restoration from a range of corruption strengths, providing the operator needed for this use. The current archive does not include a separate quantitative perturb-and-recover evaluation.

The latent Gencoder is applied recurrently to overlapping windows of a larger canvas. Averaging overlapping predictions produces its next state, and the image denoiser renders the resulting patch codes. This repeatedly uses the unconditional ReGAE on representations produced by the conditional one. Because the latent target is shifted, successful restoration is coupled to movement through image content.

A distinction between the family and a particular sampling shortcut matters here. The final saved video routine uses recurrent latent updates but renders each RGB patch with a single conditional-denoiser call from fresh noise. This is one application of the image ReGAE, not a restriction of its recurrent interface. It also means that the saved routine is not, by itself, a demonstration of a repeated pixel-space repair trajectory. Its output is configured for 1080×1920 video at 16 frames per second.

4.4. Existing observations and their scope

The author reports that local sampling learned star-field appearance without reliably recovering the source's larger organization. Modeling the latent field then produced larger

coherent regions resembling Pandora's Cluster, together with stochastic variations and a noisy visual appearance compared informally to VHS video. Occasional changes between depicted regions, described as *teleportation*, motivate a content-ambiguity interpretation: an unconditioned latent state may resemble more than one familiar source context.

The notebooks preserve decreasing training-loss traces for both instances, reproduced in Appendix E. These support optimization of the specified reconstruction and prediction objectives. They do not measure regeneration after controlled perturbations, exact recovery of the original arrangement, or performance on another dataset. Generated stills and videos from the existing runs have not yet been incorporated into this draft; their behavior is reported from the author's observations, with supporting media to be added.

The case study therefore illustrates a way to divide dataset modeling between sample-level reconstruction and relationships among learned representations. It does not establish that two levels are sufficient for every dataset, nor that visual resemblance demonstrates complete memorization. Its role is to make the general ReGAE proposal concrete through one application, with the available evidence stated separately from its broader intended uses.

5. Discussion

5.1. Regeneration as a model capability

The main distinction in ReGAE is that decoding is an operation on an evolving data state. A latent code supplies information about the desired sample, while the denoiser acts on what is currently present. Starting from noise and starting from a perturbed reconstruction invoke the same interface. Regeneration is therefore a continuation of the reconstruction process, rather than a separately defined repair network.

This is an architectural capability and a training objective, not a guarantee that every perturbation will be reversed. The result depends on the learned operator, the conditioning information, and the corruptions represented during training. In a conditional instance, retaining the code helps specify which sample should be restored. In an unconditional instance, the current state is the only sample-specific cue, so restoration can lead to a different plausible configuration when information has been lost. This distinction is relevant to the Gencoder's reported transitions between visually ambiguous regions.

5.2. Prediction targets and sampling rules

The same sample coordinates can represent a desired state, a noise vector, or a velocity, but those quantities play different roles in an update. ReGAE predicts a corrected state and assigns it directly to the next iterate. A common DDPM parameterization predicts noise [8]. However, diffusion is not restricted to noise prediction: Chan's tutorial explicitly develops both signal and noise estimates [3]. For a corruption model

$$u_t = a_t x_{\text{clean}} + b_t \epsilon, \tag{9}$$

the estimates are related, when $a_t \neq 0$, by

$$\widehat{x}_{\text{clean}}(u_t, t) = \frac{u_t - b_t \widehat{\epsilon}_\theta(u_t, t)}{a_t}. \tag{10}$$

This algebraic conversion does not make training objectives identical without accounting for their weights. A diffusion sampler generally combines its prediction with the current state and schedule-dependent coefficients, and may add noise. Predicting the clean state does not require replacing the current state by that estimate at every step.

Flow Matching instead regresses a velocity field along probability paths [10]. Sampling integrates

$$\frac{du(t)}{dt} = v_\theta(u(t), t). \tag{11}$$

For example, an Euler update is $u(t + h) \approx u(t) + h\, v_\theta(u(t), t)$. Solvers need not be complicated: the distinction is whether the learned output is used as the next state, converted through a scheduled transition, or integrated as a derivative. These choices are summarized in Table 1. Direct state prediction is shared with other denoising formulations; the ReGAE construction combines it with repeated output reuse and absence of a timestep input.

5.3. One correction map and the landscape intuition

At fixed z, $\Delta_z(u)$ assigns a correction to the state itself, without knowing how many updates preceded it. Generation and repair therefore traverse the same learned map. In a time-conditioned construction, a generic update field $U(u, t)$ can instead assign different directions to the same u at different times. This family of fields is commonly represented by a single neural network with shared weights; it need not be a collection of separate models.

Figure 2 makes the distinction visible. Noise or time conditioning indexes different fields over the *same* state domain. It does not divide that domain into disjoint spatial bands, and time can be continuous [19]. Likewise, omitting time does not itself establish continuity: that is a property of the network's operations, and time-conditioned fields can also be continuous. The useful interpretation of one uninterrupted landscape is one correction map jointly trained across sampled corruption levels.

The basin interpretation is useful when a desired reconstruction is an attracting fixed point. It is not a consequence of omitting a clock. A learned vector field need not be the gradient of any scalar energy, and generative transport need not minimize an objective until reaching a stationary state. In particular, a diffusion or flow trajectory's terminal sample need not be an equilibrium of its field. For ReGAE,

Representative formulation	Learned output	Use in sampling
ReGAE	Desired state $D_\theta(u, z)$	Direct assignment: $u_{\text{next}} = D_\theta(u, z)$
Diffusion, noise parameterization	Noise estimate $\widehat{\epsilon}_\theta(u, t)$	Apply the schedule-dependent transition rule
Diffusion, state parameterization	Clean-state estimate $\widehat{x}_{\text{clean}}(u, t)$	Apply the schedule-dependent transition rule
Flow Matching	Velocity $v_\theta(u, t)$	Integrate the field; Euler uses $u_{\text{next}} = u + h\, v_\theta(u, t)$

Table 1. Prediction targets and update rules are separate design choices [3, 8, 10]. The diffusion rows are alternative parameterizations, not an exhaustive taxonomy; score and other parameterizations also exist. Content conditioning can be supplied in any row. The ReGAE code z is distinct from a time or noise-level input t.

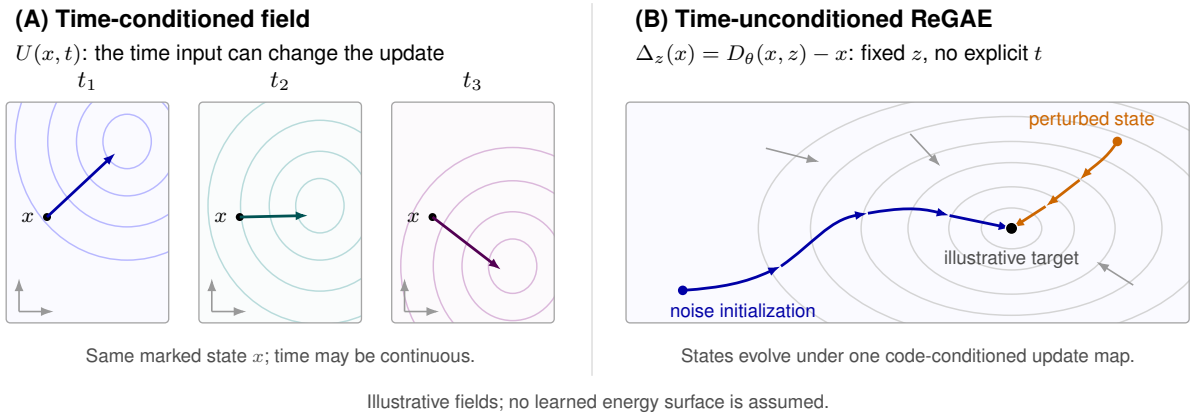


Figure 2. Conceptual comparison of time dependence. (A) Selected times index fields over the same state coordinates; the marked state can receive different updates. The panels are time slices, not disjoint regions of state space. (B) A fixed-code ReGAE uses one induced displacement field for both noise initialization and a perturbed state. The contours and paths are constructed illustrations, not measured trajectories, learned energies, or evidence of convergence. The comparison concerns the learned operator; an external sampling schedule can still introduce time dependence.

local contraction would be a sufficient mechanism for repair; Appendix A states that condition and an explicit Euler connection as mathematical interpretations, not measured properties.

The distinction between the neural map and the whole sampler also matters. The saved Pandora video routine varies its injected noise with frame index and averages overlapping predictions. Its denoisers do not receive that index, but the resulting stochastic canvas update still depends on time through the external schedule. Moreover, the shifted Gencoder target encourages progression between views, so its intended behavior is not convergence to a single unchanging image.

5.4. Modeling representations with the same kind of model

A ReGAE can model a dataset's samples, and another ReGAE can model the resulting representations. Reusing the denoiser family at both levels expresses a hypothesis that reconstruction is a useful operation across different levels of abstraction. The higher level need not be spatial: it should model whatever structure is meaningful in its input representations.

This motivates recursive or fractal organizations of models. The present experiment instantiates two separately trained members of the family, without sharing their weights. It models an encoder's output fields, not the lower model's complete internal computation or learning algorithm. Architectural reuse thus provides a concrete starting point for that broader motivation; it does not by itself demonstrate meta-learning, self-modeling, or general intelligence. The number of levels and the relationships learned at each remain design questions.

5.5. Learning and compression

The memorization question can be restated as a question about where dataset information is stored. Shared parameters describe regularities that recur across samples; individual latent tensors describe sample-specific variation. A further model can learn regularities among those tensors, reducing the relational information that must be supplied explicitly when generating plausible data. In Pandora's Cluster, that

information includes spatial organization. In another dataset, it could concern a different distribution or relationship.

This is a representational compression argument, not yet a claim about a complete image codec. A small code must be interpreted together with the model that decodes it, its numerical precision, and the fidelity of reconstruction. For the present configuration, 3,072 RGB channel values become 256 latent scalars. The factor of 12 in scalar count becomes a factor of three when comparing 8-bit RGB with 32-bit latent values, before including network storage. Appendix D provides the corresponding payload accounting.

A minimum-description-length view includes both model and data terms [14]. A practical codec additionally needs a coding procedure and a rate–distortion evaluation [6]. Learning a generator over codes does not prove that the exact original code arrangement can be discarded without loss. ReGAE provides a concrete setting for studying successive levels of regularity, while the broader intelligence-as-compression interpretation remains a subject for a companion essay.

5.6. Multiplicative gain in the default implementation

The denoiser used in the proof of concept includes a multiplicative feature branch called *gravity*. It amplifies or attenuates features through a learned gain; the term is an analogy, not a physical law or a requirement of the ReGAE family. The earlier exponential gate allowed unbounded gain. The author recalls stars becoming progressively brighter followed by dark regions, described as *neural black holes*. The associated recordings and checkpoints were not retained.

An isolated gate-only recurrence $h_{t+1} = h_t \exp(g_t)$ accumulates gain as $h_T = h_0 \exp(\sum_{t<T} g_t)$, illustrating a possible amplification mechanism. This does not establish the cause of the reported darkening or imply an implemented reset rule. The later bounded activation caps this individual gain. It does not prove stability of the full recurrent network. The precise gate and its placement are documented with the application architecture in Appendix B.

5.7. Scope and next steps

The general design accommodates conditional and unconditional modeling of one-, two-, or three-dimensional data; the evidence presented here concerns one two-dimensional visual proof of concept. Controlled regeneration measurements, architecture and loss comparisons, and applications to other data types are not part of the current report. One-step denoising training also does not explicitly optimize long-horizon behavior.

The next revision can strengthen the presentation using the existing stills, videos, checkpoints, and run records. No new experiment is required to make the family definition and the reported application precise. Further empirical evaluation can test how broadly the regenerative operator transfers across data representations, which properties are retained at each level, and how reliably perturbations are repaired.

6. Conclusion

ReGAE combines the autoencoding lineage of NCAM with the regeneration objective associated with NCA: after a state is perturbed, the same learned process can be applied to restore it. Its denoiser predicts the desired state directly, with matching input and output shapes and no required noise-level or timestep input. This yields one learned map per fixed code, whose induced displacement field provides a precise interpretation of the landscape intuition. Diffusion and flow models offer closely related descriptions of motion in sample space, but prediction targets and sampling rules must be compared separately. Pandora's Cluster illustrates how conditional and unconditional ReGAE instances can model samples and relationships among their representations. The broader question is how such reusable correction maps capture different aspects of a dataset, and under what conditions their trajectories provide reliable generation and repair.

Data and code. The original notebooks and manuscript sources are available in the Neural Pandora's Cluster repository. The source image is credited to NASA, ESA, CSA, Ivo Labbe (Swinburne), and Rachel Bezanson (University of Pittsburgh), with image processing by Alyssa Pagan (STScI) [12]. The experiments use the rendered image as a visual learning problem; generated outputs are not astronomical observations.

References

[1] Guillaume Alain and Yoshua Bengio. What regularized auto-encoders learn from the data-generating distribution. *Journal of Machine Learning Research*, 15(110):3743–3773, 2014. 2

[2] Yoshua Bengio, Li Yao, Guillaume Alain, and Pascal Vincent. Generalized denoising auto-encoders as generative models. In *Advances in Neural Information Processing Systems*, 2013. 2

[3] Stanley H. Chan. Tutorial on diffusion models for imaging and vision, 2024. Version 3, revised January 2025. 3, 6, 7

[4] Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, 2018. 3

[5] Grégoire Delétang, Anian Ruoss, Paul-Ambroise Duquenne, Elliot Catt, Tim Genewein, Christopher Mattern, Jordi Grau-Moya, Li Kevin Wenliang, Matthew Aitchison, Laurent Orseau, Marcus Hutter, and Joel Veness. Language modeling is compression. In *International Conference on Learning Representations*, 2024. 3

[6] Emilien Dupont, Adam Goliński, Milad Alizadeh, Yee Whye Teh, and Arnaud Doucet. COIN: COmpression with Implicit Neural representations. *arXiv preprint arXiv:2103.03123*, 2021. 3, 8

[7] Alejandro Hernandez, Armand Vilalta, and Francesc Moreno-Noguer. Neural cellular automata manifold. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10020–10028, 2021. 1, 2

[8] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, 2020. 3, 6, 7

[9] Justin Johnson, Alexandre Alahi, and Li Fei-Fei. Perceptual losses for real-time style transfer and super-resolution. In *European Conference on Computer Vision*, 2016. 2, 5

[10] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. In *International Conference on Learning Representations*, 2023. 3, 6, 7

[11] Alexander Mordvintsev, Ettore Randazzo, Eyvind Niklasson, and Michael Levin. Growing neural cellular automata. *Distill*, 2020. 1, 2

[12] NASA. Pandora's cluster (NIRCam image). NASA Science, 2023. Released February 15, 2023. 5, 8

[13] Konpat Preechakul, Nattanat Chatthee, Suttisak Wizadwongsa, and Supasorn Suwajanakorn. Diffusion autoencoders: Toward a meaningful and decodable representation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. 3

[14] Jorma Rissanen. Modeling by shortest data description. *Automatica*, 1978. 3, 8

[15] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. 3

[16] Tamar Rott Shaham, Tali Dekel, and Tomer Michaeli. SinGAN: Learning a generative model from a single natural image. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019. 3

[17] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations*, 2015. 2, 5

[18] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021. 3

[19] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations*, 2021. 3, 6

A. Mathematical Interpretations of the Direct Map

This appendix records sufficient conditions and algebraic interpretations. None is claimed to have been measured for the archived networks.

A.1. Hidden corruption level under squared error

Fix an encoder and let X be a clean training sample, $Z = E_\phi(X)$ its code, A a sampled corruption strength, and $\widetilde{X} = C_A(X, \epsilon)$ the observed state. Under squared-error reconstruction, an unconstrained optimal predictor is the conditional mean

$$D^*(u, z) = \mathbb{E}[X \mid \widetilde{X} = u, Z = z]. \tag{12}$$

Define $\mu_\alpha(u, z)$ as the corresponding conditional mean with $A = \alpha$ also observed. The law of total expectation gives

$$D^*(u, z) = \int \mu_\alpha(u, z)\, P(d\alpha \mid u, z). \tag{13}$$

Here $P(d\alpha \mid u, z)$ is the posterior law of A, allowing discrete, continuous, or mixed corruption schedules. Thus omitting the level label entails resolving uncertainty about that level from the state and code. It is a posterior-weighted combination, not uniform averaging of timestep-specific predictors. If the observation is ambiguous, a single map can compromise among compatible explanations. This derivation assumes squared error and an unrestricted predictor; the implemented perceptive loss and latent L_1 loss do not automatically yield this conditional-mean formula. It establishes neither a score identity nor an evaluation of the trained networks.

A.2. A sufficient local repair condition

Write $D_z(u) = D_\theta(u, z)$ and suppose u^* is a fixed point. If a neighborhood $\mathcal{B}$ is invariant under D_z and, for some $0 \le q < 1$,

$$\|D_z(u) - u^*\| \le q\|u - u^*\|, \qquad u \in \mathcal{B}, \tag{14}$$

then repeated application gives $\|u_k - u^*\| \le q^k\|u_0 - u^*\|$. In this hypothetical local regime, perturbations that remain in $\mathcal{B}$ are repaired. The scalar $V(u) = \frac{1}{2}\|u - u^*\|^2$ also decreases, since $V(D_z(u)) \le q^2 V(u)$. This provides one precise meaning for a basin and its contour lines. Training for reconstruction alone does not prove the condition; the gate bound likewise supplies no contraction bound for the full network.

A.3. An algebraic Euler connection

For any direct map, define $\Delta_z(u) = D_z(u) - u$. Euler's rule applied to the constructed autonomous ODE

$$\frac{du}{d\tau} = \Delta_z(u) \tag{15}$$

with unit step gives exactly $u_{k+1} = D_z(u_k)$. This identifies the discrete update algebraically; it does not establish that a unit step accurately resolves a continuous trajectory. The network is not trained as an ODE velocity or a probability-flow score by this rewriting. In particular, no conservative-field assumption, monotonic energy descent, or agreement with a prescribed probability path follows from it. The implemented operation remains direct state assignment.

B. Pandora Implementation Details

This appendix specifies the archived v2 proof of concept. Its dimensions, corruption schedule, losses, shifted grids, and video routine are application choices within the family defined in Sec. 3. In particular, the final routine repeatedly updates latent states while using one image-denoiser call per rendered patch. That shortcut does not define or limit the recurrent conditional interface.

B.1. Patch representation and denoising objective

Let $x \in [-1, 1]^{3\times p\times p}$ be a clean RGB patch, with $p = 32$. The encoder produces $z = E_\phi(x) \in \mathbb{R}^d$, where $d = 256$. Patches are sampled at random locations in the source image. A convolutional inlet is followed by four spatial downsampling stages, giving $256 \times 2 \times 2$ features. After flattening, two residual multilayer perceptron blocks map $1024 \to 256 \to 256$ to produce z. Its output is not constrained to unit norm, and training imposes no Gaussian-prior or vector-quantization objective.

We define the corruption operator

$$C_\alpha(x, \epsilon) = (1 - \alpha)x + \alpha\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I). \tag{16}$$

The implementation samples k uniformly from $\{0, \dots, 7\}$ and u uniformly from $[0, 1)$, then sets $\alpha = f(k + u)$, where

$$f(s) = \mathrm{clip}_{[0,1]}\left(\frac{1.5^{-s} - \rho}{1 - \rho}\right), \qquad \rho = 0.0586. \tag{17}$$

Thus training includes nearly clean inputs and inputs dominated by noise. Within each batch, the second half reuses the first half's noise with opposite sign and the same corruption coefficients, paired with separately sampled clean patches. Neither k nor α is supplied explicitly to the network. This linear mixture is a corruption schedule, not an assertion of the variance-preserving forward process used in standard diffusion formulations.

The jointly optimized encoder and decoder minimize

$$\mathcal{L}_{\mathrm{AE}} = \mathbb{E}_{x,\alpha,\epsilon}\left[\ell_{\mathrm{perc}}\big(D_\theta(C_\alpha(x, \epsilon), E_\phi(x)), x\big)\right]. \tag{18}$$

Here $\ell_{\mathrm{perc}}(a, b)$ sums the mean absolute feature differences from four successive frozen VGG16 blocks. Inputs are mapped from $[-1, 1]$ to $[0, 1]$, normalized with the pretrained network's channel statistics, and bilinearly resized

to 64×64. If Φ_ℓ includes this preprocessing and the first ℓ feature blocks, the loss is

$$\ell_{\text{perc}}(a, b) = \sum_{\ell=1}^{4} \frac{\|\Phi_\ell(a) - \Phi_\ell(b)\|_1}{n_\ell}, \tag{19}$$

where n_ℓ is the number of feature entries. Each training example requires one decoder evaluation and predicts the clean patch directly. There is no recurrent backpropagation through a sequence of regenerated images. At reconstruction time, a single call $D_\theta(\epsilon, z)$ can likewise decode a patch from its code and fresh Gaussian noise.

B.2. Conditional decoder and multiplicative gate

The decoder contains a convolutional inlet, a U-shaped network with group normalization, a multiplicative feature gate, a second U-shaped network without normalization, and an output projection. Multiscale skip connections, learned patch-relative positional tensors, and attention within the processed window provide spatial interaction. Latent-dependent transformations modulate intermediate features through learned additive and multiplicative terms. Consequently, the update operates over an entire patch or latent window rather than being restricted to a fixed local cellular neighborhood.

The first U-shaped network produces two feature tensors, h and g. The v2 gate, informally called *gravity*, computes

$$\widetilde{h} = h \odot \gamma(g), \qquad \gamma(g) = \text{clip}_{[0,2]}(1 + g/3) + 10^{-4}. \tag{20}$$

Its multiplier therefore lies in $[10^{-4}, 2 + 10^{-4}]$. The earlier v0 decoder instead used $\widetilde{h} = h \odot \exp(g)$. The replacement bounds this particular gain; it does not impose a global activation bound or prove stability under repeated updates. Subsequent transformations can still amplify features. Both D_θ and the v2 Gencoder use this gated architecture, but the latter receives no separate conditioning vector.

B.3. Shifted latent grids

Let the source image have height H and width W, and define $H_z = \lfloor H/p \rfloor$ and $W_z = \lfloor W/p \rfloor$. With the first stage fixed, we construct $K = 4$ offset grids $Z^{(o)} \in \mathbb{R}^{d \times H_z \times W_z}$, $o \in \{0, \ldots, K-1\}$. To match the actual crop helper, their pixel origins are

$$r_o(i) = \left\lfloor \frac{(i + o/K)(H - p)}{H_z} \right\rfloor, \tag{21}$$

$$c_o(j) = \left\lfloor \frac{(j + o/K)(W - p)}{W_z} \right\rfloor. \tag{22}$$

The entry $Z^{(o)}_{:,i,j}$ encodes the $p \times p$ crop starting there. Nonzero-offset maps omit their final row and column during construction; training windows are selected to avoid these unfilled entries. This convention approximates a regular patch lattice but is not exactly $r_o(i) = pi + po/K$, owing to image dimensions and integer rounding.

For a $q \times q$ window with $q = 32$, sample an offset o and a valid origin (i, j). Set $o' = (o + 1) \bmod K$ and $b = \lfloor (o + 1)/K \rfloor$. The paired windows are

$$z^- = Z^{(o)}_{:,i:i+q,\, j:j+q}, \tag{23}$$

$$z^+ = Z^{(o')}_{:,i+b:i+b+q,\, j+b:j+b+q}. \tag{24}$$

The wrap at $o = K - 1$ advances the integer lattice origin, maintaining a positive displacement in both source-image axes. Each target code describes a shifted pixel patch, rather than copying the overlapping input tokens and predicting only a newly exposed border. The displacement is approximately $p/K = 8$ pixels per axis, with rounding-dependent variation.

The Gencoder minimizes the elementwise mean absolute error

$$\mathcal{L}_G = \mathbb{E}\left[\frac{\|G_\psi(C_\alpha(z^-, \epsilon)) - z^+\|_1}{dq^2} \right], \tag{25}$$

using the same corruption schedule and sign-paired noise construction. Encoder and image-decoder parameters are not optimized in this stage. A latent window decodes into $pq \times pq = 1024 \times 1024$ pixels. Its source-image support instead spans $r_o(i+q-1) - r_o(i) + p$ rows and the analogous number of columns: the rounded grid makes this slightly smaller than the decoded extent. Global source-image coordinates and the offset index are absent from the Gencoder's inputs, although its internal positional tensors distinguish locations within a window. Spatial progression must therefore be inferred from the available content.

B.4. Optimization

Both stages use Adam with AMSGrad, an initial learning rate of 5×10^{-5}, moment coefficients $(0.5, 0.9)$, numerical stabilizer 10^{-8}, and no weight decay. The nominal batch contains 64 examples. The first stage is configured for 256 epochs, halving its learning rate every 64 epochs and increasing gradient accumulation from one to two batches at epoch 128. The second stage is configured for 1,024 epochs, increasing accumulation from one through four batches at 256-epoch intervals. Accumulated losses are divided by the number of contributing batches before backpropagation.

An epoch is a bookkeeping interval, not an exhaustive pass over disjoint crops. Both stages define its length as $\lfloor H_z W_z / 64 \rfloor$, or 3,607 updates for the recorded image dimensions, while drawing training locations randomly. In particular, a Gencoder epoch is not normalized by the number of latent sites in each large window. The frozen VGG features provide the first stage's perceptive objective; they are not an additional trainable branch. No multi-step rollout loss couples successive sampled transitions during optimization.

B.5. Overlapping recurrent generation

The final video routine maintains a latent canvas $S_t \in \mathbb{R}^{d\times A\times B}$ with $A = \lceil 1080/p \rceil = 34$ and $B = \lceil 1920/p \rceil = 60$. It initializes S_0 with Gaussian noise. Its optional seven-step initialization loop is commented out; generation begins directly with the recurrent update below.

Let $\mathcal{P}$ be the multiset of $q \times q$ window origins produced by integer-rounded evenly spaced coordinates. In this configuration the row sequence is $(0, 0, 1, 2)$ and the column sequence is $(0, 9, 18, 28)$. Repeated origins are retained. Write W_s for window extraction and $W_s^\top$ for placing a window into a zero canvas. The coverage count is $N = \sum_{s\in\mathcal{P}} W_s^\top \mathbf{1}$, broadcast across latent channels.

Noise refresh is stronger near the bottom and right edges. For canvas coordinates (r, c), define

$$\delta(r, c) = \min(A - 1 - r, B - 1 - c), \tag{26}$$

$$m(r, c) = \min(6, 4 + \delta(r, c)), \tag{27}$$

$$a_t(r, c) = (1 + \beta_t) f(m(r, c)), \tag{28}$$

$$\beta_t = \tfrac{1}{2}\left[1 + \cos(t/1824)\right]. \tag{29}$$

The denominator is the configured 114 seconds times 16 frames per second; it is the cosine's argument scale, not a full-period duration. Extending C to elementwise masks, the synchronous update is

$$S_{t+1} = N^{-1} \odot \sum_{s\in\mathcal{P}} W_s^\top G_\psi(C_{W_s a_t}(W_s S_t, \eta_{t,s})), \tag{30}$$

with independent Gaussian $\eta_{t,s}$ for each window evaluation. Every prediction reads the previous canvas; an initially zero accumulator collects predictions before normalization by coverage. Earlier exploratory routines also used overwrite-based overlap handling, but Eq. (30) describes the final averaging routine.

There is no explicit spatial roll, coordinate update, or translated copy in this loop. Motion is learned through Eq. (24). If the model tracks a consistent source region, advancing the viewed crop down and right corresponds to apparent image-content motion up and left. That interpretation is conditional on coherent tracking, not a guaranteed trajectory.

Each displayed frame is then generated independently in RGB space. For every latent location v, the routine draws fresh pixel noise $\xi_{t,v}$ and computes

$$\widehat{x}_{t,v} = Q(D_\theta(\xi_{t,v}, S_{t+1,:,v})), \tag{31}$$

where Q clips to $[-1, 1]$ and quantizes to 256 levels. Nonoverlapping decoded patches form a 1088×1920 image, centrally cropped to 1080×1920. The previous RGB frame is never fed back: latent refresh and fresh decoder noise are separate stochastic sources. The configured output is 7,280 frames at 16 frames per second, or 7 minutes 35 seconds; this specifies the routine, not evidence that its saved progress widget records completion.

Implementation qualifications, including the historical learning-rate schedule and earlier sampling cells, are documented in Appendix C.

C. Archive and Implementation Details

Source of the technical description. The principal implementation is the v2 notebook, while v0 documents architectural and sampling development. The repository preserves both original files in its notebooks directory, together with their SHA-256 hashes. The description in this manuscript was obtained by inspecting code and saved outputs. No notebook training or generation cell was executed to prepare this draft. The separate NVD notebooks are not evidence for the ReGAE results.

Stage boundaries. The v2 configuration jointly trains the image encoder and decoder, then constructs latent maps for the Gencoder. The latter is a separately trained instance of the decoder family with 256 input/output channels and no external conditioning code. The image decoder uses three input/output channels and latent conditioning. Shared architecture thus refers to the computational construction, not tied weights. The patch-relative positional parameters do not encode global source-image coordinates.

Optimization and reproducibility. The recorded environment reports PyTorch 2.2.2 with CUDA 12.1, and the code sets a random seed of 4202. These facts do not specify the original GPU, elapsed training time, or complete software environment. The archive contains configured training loops and saved displays, but no complete execution log establishing every historical state. In v2, the Gencoder loop computes a decaying learning rate but assigns it to the AE optimizer, leaving the Gencoder optimizer unchanged by that schedule. This is preserved as a historical qualification rather than silently corrected. Exact reproduction should distinguish a faithful historical configuration from a cleaned implementation.

Exploratory sampling in v0. The earlier notebook includes source-code reconstruction, latent generation, random overlapping patch updates, repeated local refinement, and shifted raster sweeps. Some updates protect previously completed pixels or use update-budget masks. Their motivation includes an analogy to wave-function-collapse-style synthesis. The cells do not implement a documented entropy-ranked constraint-propagation algorithm, and their current channel and conditioning assumptions are not mutually consistent with all retained network definitions. We therefore avoid presenting them as a verified benchmark of sampling

Representation or parameter payload	MB
Source RGB array, 8 bits per channel	710.45
One $419 \times 551 \times 256$ grid, FP32	236.41
Image encoder and denoiser, FP32	222.64
Gencoder, FP32	341.92
Image denoiser plus one grid, FP32	440.36
AE and Gencoder parameters, FP32	564.56

Table 2. Calculated uncompressed payloads for the Pandora setup. Rows describe different accounting choices, not interchangeable complete codecs. Optimizer state and file overhead are excluded. The four shifted training grids occupy approximately 945.64 MB in FP32.

strategies. The later v2 synchronous overlap average is specified explicitly in Eq. (30).

Extent of a latent window. The decoder tiles 32×32 generated image patches into a 1024×1024 macro-image. The source crops used to build a corresponding training window are positioned by the rounded lattice in Eq. (22), rather than exactly 32 pixels apart. "Megapixel window" consequently describes the decoded tile extent; it does not assert that all sampled training windows form an exact, gap-free partition of the source image.

Visual evidence to add from existing runs. The most informative additions would be a source region alongside its patch-code reconstruction, a spatially coherent Gencoder sample, and a short sequence showing a typical trajectory and a reported jump. Each should identify its checkpoint, sampler configuration, and whether its RGB rendering uses fresh noise. The unstable exponential-gate example cannot currently be illustrated from an original recording and should remain identified as an anecdote. This list specifies documentation needs, not new experiments or figures that already exist in this draft.

D. Storage Accounting for the Proof of Concept

The stored parameter counts are 4,673,752 for the encoder, 50,986,296 for the conditional image denoiser, and 85,479,680 for the Gencoder. These counts describe this implementation, not a required model size. Table 2 calculates uncompressed payloads in decimal megabytes. A code grid alone does not specify a complete codec for the image, including its boundary conventions and reconstruction distortion.

AMSGrad stores three moment-related tensors per parameter in addition to the weights. Including these tensors in FP32 can bring training checkpoints near the author's recalled 0.89 GB for the AE and 1.4 GB for the Gencoder. This is an inference from the optimizer and parameter counts, not a measurement of the unavailable checkpoint files. A storage comparison with JPEG or another codec would also need quantization, coding, model costs, and matched reconstruction distortion.

E. Preserved Training Plots

The plots reproduce unchanged PNGs from the archived v2 outputs. Their horizontal axes count optimizer updates despite the inherited "epoch" label. Vertical values are square roots of different training objectives and are not directly comparable.

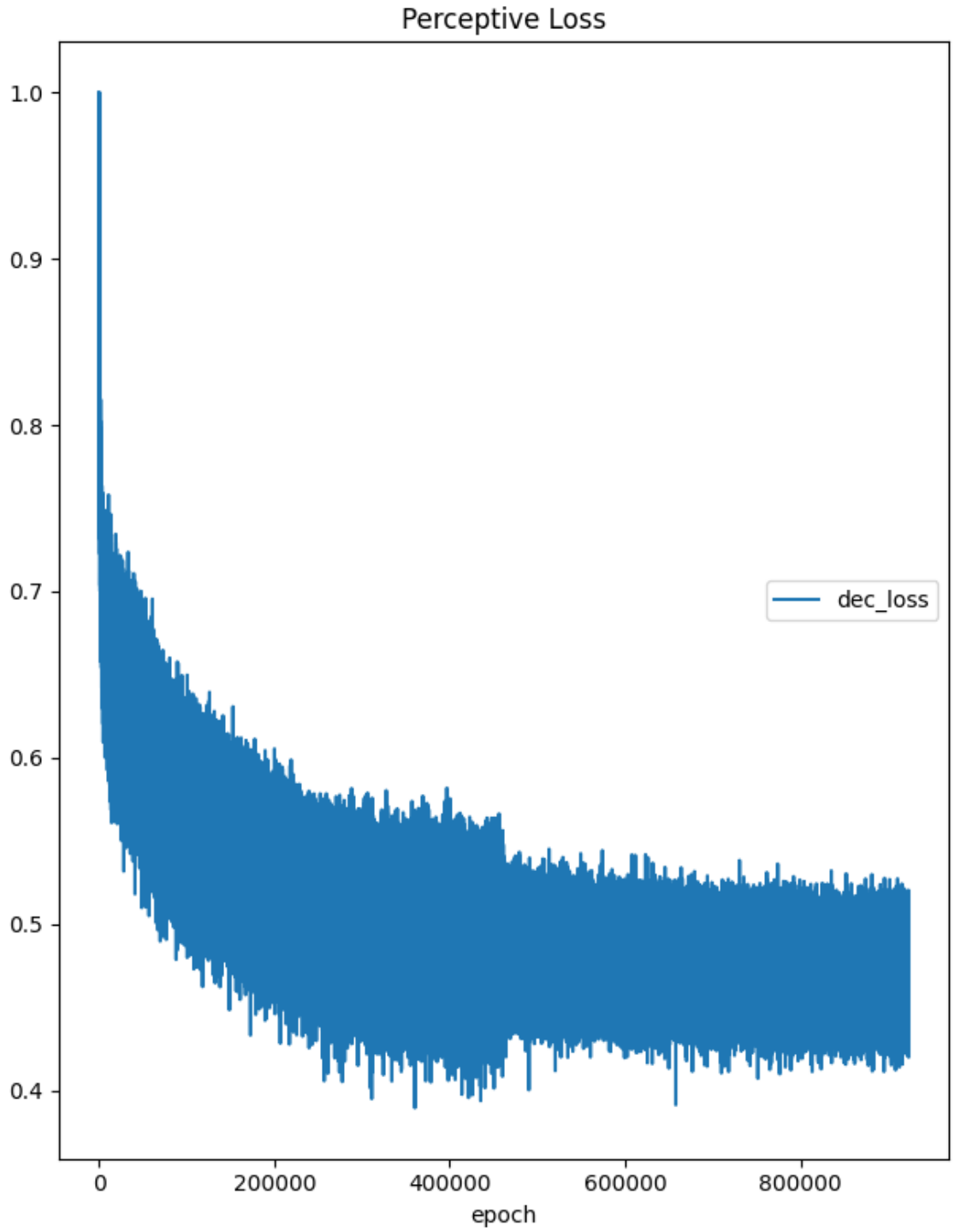


Figure 3. Saved image-autoencoder training display (v2, code cell 39 under zero-based notebook indexing). The summary reports minimum 0.389, current 0.450, and maximum 1.000; the history was clipped to $[0, 1]$ for plotting. The displayed quantity is the square root of the VGG feature L_1 loss. The history supports optimization of the training objective, not a held-out quality estimate.

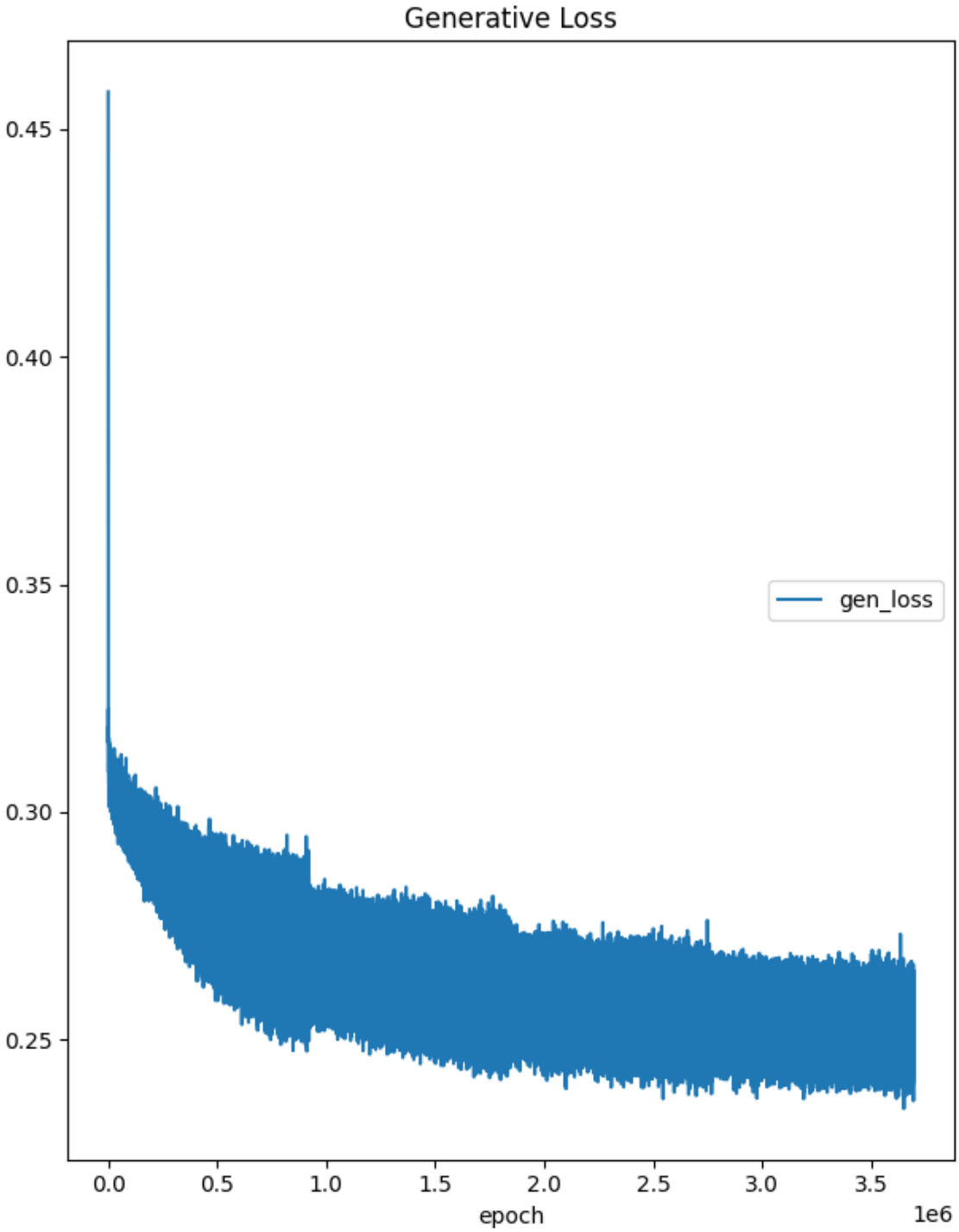


Figure 4. Saved Gencoder training display (v2, code cell 46 under zero-based notebook indexing). The summary reports minimum 0.235, current 0.252, and maximum 0.458. The displayed quantity is the square root of the latent L_1 prediction loss. No new training was run to produce this figure.